

การโปรแกรมเบื้องต้น ภาษาไพธอน

Introduction to Python Programming

None W. (Yuan)

สารบัญ (Outline)

- 1 ชนิดข้อมูลและตัวแปร (Data Types & Variables)
- 2 ตัวดำเนินการ (Operators)
- 3 การแสดงผล (Output)
- 4 เงื่อนไข (Conditionals)
- 5 การวนซ้ำ (While Loop)
- 6 แวลลำดับ (1D Array / List)
- 7 สตริง (String)
- 8 ตัวอย่างประยุกต์
- 9 สรุป

ก่อนเรียน (Prerequisites)

สิ่งที่ควรรู้ก่อนเรียน

- **คณิตศาสตร์พื้นฐาน** — การบวก ลบ คูณ หาร, การเปรียบเทียบ, ตรรกศาสตร์
- **ตัวแปร** — แนวคิดเรื่องการเก็บค่า (เหมือนคณิตศาสตร์)
- **คอมพิวเตอร์** — การใช้งานเครื่องคอมพิวเตอร์พื้นฐาน

การเขียนโปรแกรมคืออะไร?

การเขียนโปรแกรมคือการ **สั่งให้คอมพิวเตอร์ทำงานทีละขั้นตอน** โดยใช้ภาษาที่คอมพิวเตอร์เข้าใจ — ในที่นี้คือ **ภาษาไพธอน (Python)**

ชนิดข้อมูลและตัวแปร (Data Types & Variables)

ชนิดข้อมูลพื้นฐาน (Basic Data Types)

4 ชนิดข้อมูลสำคัญ

- `int` — จำนวนเต็ม (integer)
 - เช่น 5, -3, 0, 100
- `float` — จำนวนจริง/ทศนิยม
 - เช่น 3.14, -0.5, 2.0
- `bool` — ค่าความจริง (Boolean)
 - True, False
- `str` — ข้อความ (string)
 - เช่น "Hello", "123"

ตัวอย่าง: การเช็คชนิดข้อมูล (ไม่ออกสอบ)

```
type(5) → <class 'int'>    type(3.14) → <class 'float'>
type(True) → <class 'bool'>  type("hello") → <class 'str'>
```

ตัวแปรและการกำหนดค่า (Variables & Assignment)

การกำหนดค่า

ตัวแปร คือชื่อที่ใช้อ้างถึงค่าที่เก็บในหน่วยความจำ

ใช้เครื่องหมาย = ในการ **กำหนดค่า**: $x = 5$ หมายถึง เก็บค่า 5 ในตัวแปร x

ตัวอย่าง: การกำหนดค่า

```
x = 5
y = 3
z = x + y      (ได้ z = 8)
name = "Yuan"
is_valid = True
```

กฎการตั้งชื่อตัวแปร

- เริ่มต้นด้วยตัวอักษร หรือ _ (underscore)
- ประกอบด้วยตัวอักษร ตัวเลข และ _
- ห้าม เริ่มต้นด้วยตัวเลข
- ห้าม ใช้คำสงวน เช่น if, while
- Case-sensitive: x กับ X เป็นคนละตัว

ตัวดำเนินการ (Operators)

ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic Operators)

เครื่องหมายพื้นฐาน

เครื่องหมาย	ความหมาย	ตัวอย่าง
+	บวก	$5 + 3 \rightarrow 8$
-	ลบ	$5 - 3 \rightarrow 2$
*	คูณ	$5 * 3 \rightarrow 15$
/	หาร	$5 / 2 \rightarrow 2.5$
//	หารปัดเศษ	$5 // 2 \rightarrow 2$
%	มอดุโล (mod)	$5 \% 2 \rightarrow 1$
**	ยกกำลัง	$2 ** 3 \rightarrow 8$

ตัวอย่าง: mod (%)

$$10 \% 3 = 1 \quad (10 = 3 \times 3 + 1)$$

$$17 \% 5 = 2 \quad (17 = 5 \times 3 + 2)$$

$$8 \% 2 = 0 \quad (8 = 2 \times 4 + 0)$$

การใช้งาน:

- เช็คเลขคู่/คี่: $n \% 2 == 0$
- หลักหน่วย: $n \% 10$
- การวนรอบ (cycle)

ตัวดำเนินการเปรียบเทียบและตรรกะ

Comparison

`==` (เท่ากับ), `!=` (ไม่เท่ากับ), `<`, `>`, `<=`, `>=`

`5 == 5 → True`

`3 != 5 → True`

Logical

`and`, `or`, `not`

`(5 > 3) and (2 < 4) → True`

`(5 > 3) or (2 > 4) → True`

`not (5 == 5) → False`

ลำดับความสำคัญของตัวดำเนินการ (Operator Precedence)

ลำดับจากสูงไปต่ำ (ทำก่อน → ทำหลัง)

- 1 `()` — วงเล็บ (สำคัญที่สุด)
- 2 `**` — ยกกำลัง (ทำจากขวาไปซ้าย)
- 3 `*`, `/`, `//`, `%` — คูณ หหาร มอดุโล
- 4 `+`, `-` — บวก ลบ
- 5 `==`, `!=`, `<`, `>`, `<=`, `>=` — เปรียบเทียบ
- 6 `not` — นิเสธ
- 7 `and` — และ
- 8 `or` — หรือ (สำคัญน้อยที่สุด)

ลำดับความสำคัญของตัวดำเนินการ (Operator Precedence)

ตัวอย่าง

$$3 + 4 * 2 = 11 \text{ (คูณก่อนบวก)}$$

$$(3 + 4) * 2 = 14 \text{ (วงเล็บก่อน)}$$

$$10 / 2 ** 2 = 2.5 \text{ (ยกกำลังก่อนหาร)}$$

$$2 ** 3 ** 2 = 512 \text{ (ขวาไปซ้าย)}$$

การแสดงผล (Output)

คำสั่ง `print()`

ใช้แสดงผลลัพธ์ออกทางหน้าจอ

`print()` — แสดงค่าแล้วขึ้นบรรทัดใหม่

ตัวอย่าง: การใช้ `print()`

```
print("Hello World")    → Hello World
```

```
print(3 + 5)           → 8
```

```
x = 10
```

```
print(x)               → 10
```

```
print("x =", x)        → x = 10
```

เงื่อนไข (Conditionals)

คำสั่ง if / elif / else

การตัดสินใจในโปรแกรม

if เงื่อนไข: ทำเมื่อเงื่อนไขเป็นจริง

elif เงื่อนไข: เงื่อนไขรอง (else if) เมื่อเงื่อนไขข้างบนไม่เป็นจริง มีหลาย elif ได้

else: ทำเมื่อไม่มีเงื่อนไขใดเป็นจริงเลย

สำคัญ!

if-elif-else เป็นกลุ่มเดียวกันจะเข้าเงื่อนไขได้แค่เงื่อนไขเดียวในกลุ่ม หลังจากเข้าเงื่อนไขด้านบนแล้วจะไม่สนใจเงื่อนไขด้านล่างอีก

ถ้าจะสร้างกลุ่มใหม่ต้องเริ่ม if ใหม่

ตัวอย่างและการเยื้อง

ตัวอย่าง: เกรด

```
score = 85
if score >= 80:
    print("A")
elif score >= 70:
    print("B")
elif score >= 60:
    print("C")
else:
    print("F")
    print("You failed!")
```

การเยื้อง (Indentation)

การเยื้อง (indent) มีความหมาย!

ในภาษา Python การเว้นวรรคหน้าคำสั่ง เป็นตัวกำหนดว่าคำสั่งนั้นอยู่ในบล็อกใด

กฎ:

- ใช้ 4 ช่องว่าง (space) หรือ 1 tab
- ห้ามปนกัน ในบล็อกเดียวกัน
- คำสั่งในบล็อกเดียวกันต้องเยื้องเท่ากัน

ตัวอย่าง: การตามรอยเงื่อนไข

โจทย์

ถ้า $x = 5$ และ $y = 12$ ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
x = 5
y = 12
if x > 10:
    print("A")
elif y > 10:
    print("B")
else:
    print("C")
```

วิธีตามรอย

$x = 5$: $x > 10$ เป็น False → ไม่ทำ
 $y = 12$: $y > 10$ เป็น True → **ทำ** elif
แสดงผล "B" (ข้าม else)

ตัวอย่าง: การตามรอยเงื่อนไข

โจทย์

ถ้า $x = 11$ และ $y = 12$ ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
x = 11
y = 12
if x > 10:
    print("A")
elif y > 10:
    print("B")
if x > 5:
    print("C")
```

วิธีตามรอย

$x = 11$: $x > 10$ เป็น True → ทำ if ด้านบน
แสดงผล "A" (ข้าม elif)
 $x = 11$: $x > 5$ เป็น True → ทำ if ด้านล่าง
แสดงผล "C"

การวนซ้ำ (While Loop)

คำสั่ง while

การทำซ้ำที่เงื่อนไขเป็นจริง

`while` เงื่อนไข: ทำคำสั่งในบล็อกซ้ำไปเรื่อย ๆ ตราบเท่าที่ เงื่อนไขยังเป็นจริง

ตัวอย่าง: นับ 1 ถึง 5

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

ผลลัพธ์: 1 2 3 4 5

การทำงาน

รอบ	i	i <= 5?
1	1	True → print 1
2	2	True → print 2
3	3	True → print 3
4	4	True → print 4
5	5	True → print 5
6	6	False → ออกจากวน

break และ continue

คำสั่งควบคุมการวนซ้ำ

`break` — ออกจาก การวนซ้ำทันที (หยุดเลย)

`continue` — ข้าม คำสั่งที่เหลือ แล้วขึ้นรอบถัดไป

ตัวอย่าง: break

```
i = 1
while True:
    print(i)
    if i == 3: break
    i = i + 1
```

ผลลัพธ์: 1 2 3

ตัวอย่าง: continue

```
i = 0
while i < 5:
    i = i + 1
    if i == 3: continue
    print(i)
```

ผลลัพธ์: 1 2 4 5

ตัวอย่าง: การตามรอย while loop

โจทย์

ถ้าเริ่มต้น $n = 10$ ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
n = 10
s = 0
while n > 0:
    s = s + (n % 10)
    n = n // 10
print(s)
```

วิธีตามรอย

รอบ	n	n%10	s
เริ่ม	10	-	0
1	10	0	0
2	1	1	1

แสดงผล 1 (ผลรวมเลขโดดของ 10 คือ $1 + 0 = 1$)

ตัวอย่าง: while loop (ต่อ)

โจทย์

ถ้า $x = 2$ และ $y = 5$ ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
x = 2
y = 5
c = 0
while x < y:
    x = x + 1
    y = y - 1
    c = c + 1
print(c)
```

วิธีตามรอย

รอบ	x	y	c
เริ่ม	2	5	0
1	3	4	1
2	4	3	2

รอบ 3: $x = 4, y = 3 \rightarrow x < y = \text{False} \rightarrow$ จบ

แสดงผล 2

Nested While Loop (while ซ้อน while)

แนวคิด

while loop ซ้อนกัน: loop ในอยู่ภายใน loop นอก
loop นอกวน 1 รอบ → loop ในวนจนครบ → loop นอกวนรอบถัดไป

ตัวอย่าง: สูตรคูณ

```
i = 1
while i <= 3:
    j = 1
    while j <= 3:
        print(i, "x", j, "=", i*j)
        j = j + 1
    i = i + 1
```

การทำงาน

i=1: j=1, 2, 3 → 1x1, 1x2, 1x3

i=2: j=1, 2, 3 → 2x1, 2x2, 2x3

i=3: j=1, 2, 3 → 3x1, 3x2, 3x3

รวม $3 \times 3 = 9$ รอบ!

ตัวอย่าง: Nested While Loop

โจทย์

ถ้า $a = 2$ และ $b = 3$ ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

```
s = 0
i = 1
while i <= 2:
    j = 1
    while j <= 3:
        s = s + i * j
        j = j + 1
    i = i + 1
print(s)
```

วิธีตามรอย

$i=1: j=1 \rightarrow s = 0 + 1$

$j=2 \rightarrow s = 1 + 2 = 3$

$j=3 \rightarrow s = 3 + 3 = 6$

$i=2: j=1 \rightarrow s = 6 + 2 = 8$

$j=2 \rightarrow s = 8 + 4 = 12$

$j=3 \rightarrow s = 12 + 6 = 18$

แสดงผล 18

แถวลำดับ (1D Array / List)

List — แกลวลำดับ 1 มิติ

นิยาม

list คือโครงสร้างข้อมูลที่เก็บค่าหลาย ๆ ค่าเรียงต่อกันในตัวแปรเดียว
สร้างด้วย [] คั่นด้วย , เช่น `arr = [10, 20, 30, 40]`

การเข้าถึงสมาชิก (Indexing)

ดัชนี (index) เริ่มจาก 0

ค่า	10	20	30	40
index	0	1	2	3
<code>arr[0]</code>	→ 10			
			<code>arr[2]</code>	→ 30

ตัวอย่าง: list

```
arr = [10, 20, 30, 40]
print(arr[0]) # 10
arr[2] = 99
print(arr) # [10, 20, 99, 40]
print(len(arr)) # 4
```

ตัวอย่าง: การวน loop กับ list

โจทย์

ถ้า `arr = [3, 7, 2, 8, 5]` ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
arr = [3, 7, 2, 8, 5]
s = 0
i = 0
while i < len(arr):
    if arr[i] % 2 == 1:
        s = s + arr[i]
    i = i + 1
print(s)
```

วิธีตามรอย

`arr[0] = 3:  $3 \% 2 = 1$  (คี่)  $\rightarrow s = 3$`
`arr[1] = 7: คี่  $\rightarrow s = 10$`
`arr[2] = 2: คู่  $\rightarrow$ ข้าม`
`arr[3] = 8: คู่  $\rightarrow$ ข้าม`
`arr[4] = 5: คี่  $\rightarrow s = 15$`

แสดงผล 15

สตริง (String)

String — ข้อความ

นิยาม

str คือข้อมูลประเภทข้อความ สร้างด้วย " " หรือ ' '
String ก็คือ **list ของตัวอักษร** — ใช้ index ได้เหมือน list

ตัวอย่าง: string

```
s = "PYTHON"
print(s[0]) # P
print(s[2]) # T
print(len(s)) # 6
a = "Hello"; b = "World"
print(a + " " + b) # Hello World
```

การดำเนินการกับ string

- `len(s)` — ความยาว
- `s[i]` — ตัวอักษรที่ index `i`
- `s1 + s2` — ต่อข้อความ
- `s * 3` — ทำซ้ำ
(`"Hi"*3` → `"HiHiHi"`)

ตัวอย่าง: การวน loop กับ string

โจทย์

ถ้า `s = "COMPUTER"` ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
s = "COMPUTER"
c = 0
i = 0
while i < len(s):
    if s[i] == 'U': break
    c = c + 1
    i = i + 1
print(c)
```

วิธีตามรอย

```
s[0] = 'C' → c = 1
s[1] = 'O' → c = 2
s[2] = 'M' → c = 3
s[3] = 'P' → c = 4
s[4] = 'U' → เจอ 'U', break!
```

แสดงผล 4 (จำนวนตัวอักษรก่อนถึง U)

ตัวอย่างประยุกต์

ตัวอย่าง: การนับเลขโดด

โจทย์

ถ้า $n = 12345$ ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
n = 12345
c = 0
while n > 0:
    c = c + 1
    n = n // 10
print(c)
```

วิธีตามรอย

รอบ	n	c
เริ่ม	12345	0
1	1234	1
2	123	2
3	12	3
4	1	4
5	0	5

แสดงผล 5 (จำนวนเลขโดด)

ตัวอย่าง: การหาเลขคู่

โจทย์

ถ้า `arr = [4, 7, 1, 8, 3]` ผลลัพธ์ของโปรแกรมเป็นเท่าใด?

โปรแกรม

```
arr = [4, 7, 1, 8, 3]
c = 0
i = 0
while i < 5:
    if arr[i] % 2 == 0:
        c = c + 1
    i = i + 1
print(c)
```

วิธีตามรอย

$4 \% 2 = 0 \rightarrow c = 1$

$7 \% 2 = 1 \rightarrow$ ข้าม

$1 \% 2 = 1 \rightarrow$ ข้าม

$8 \% 2 = 0 \rightarrow c = 2$

$3 \% 2 = 1 \rightarrow$ ข้าม

แสดงผล 2 (มีเลขคู่ 2 ตัว)

สรุป

สิ่งที่ได้เรียนรู้

- ชนิดข้อมูล: `int`, `float`, `bool`, `str`
- ตัวแปร: `x = 5`, กฎการตั้งชื่อ
- ตัวดำเนินการ: `+` `-` `*` `/` `//` `%` `**`
- เปรียบเทียบ: `==` `!=` `<` `>` `<=` `>=`
- ตรรกะ: `and` `or` `not`
- ลำดับความสำคัญ: วงเล็บ $\rightarrow$ `**` $\rightarrow$ `*/%` $\rightarrow$ `+-`
- `print()`: แสดงผลลัพธ์
- `if/elif/else`: เงื่อนไข + การเยื้อง
- `while`: วนซ้ำตามเงื่อนไข
- `break/continue`: ควบคุมการวนซ้ำ
- `list`: `arr[i]`, `len(arr)`
- `string`: `s[i]`, `len(s)`, `s1 + s2`

เอกสารนี้เป็นส่วนหนึ่งของเนื้อหาเตรียมสอบ สอวน. คอมพิวเตอร์